

18 Filebeschreibung

Um das Verständnis für die folgenden Files zu stärken, wird anhand Abbildung 29 der Gesamtablauf einer Abfrage erklärt.

Es gibt zwei virtuelle Maschinen wobei lediglich die Simulation des DNA Centers auf dem einem Server während der IPA erarbeitet wurde.

- Die Inventory Datenbank befindet sich auf dem virtuellen Server namens «inventarmichi» und stellt die Seite des Users dar. (nicht Produkt der IPA)
- Die DNASim Datenbank befindet sich auf dem virtuellen Server namens «michi» und stellt das simulierte DNA Center dar. (Produkt der IPA)

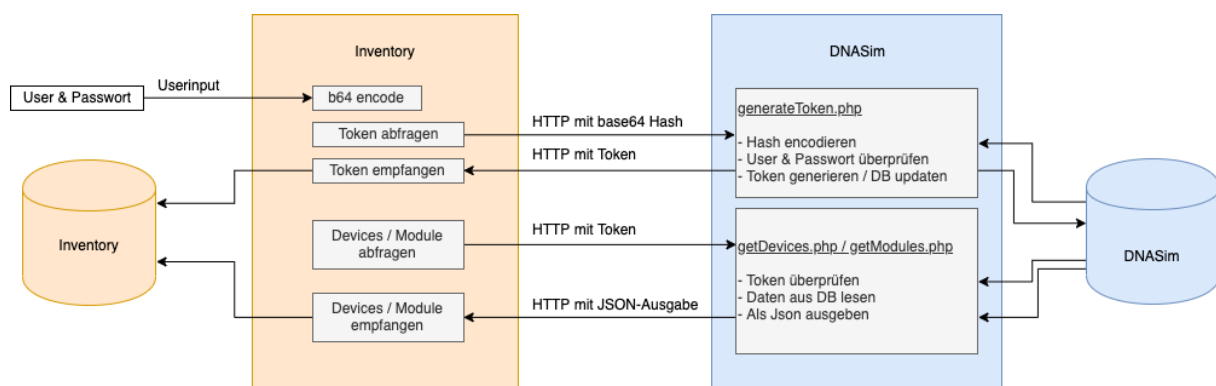


Abbildung 29: Gesamtablauf einer Abfrage

Als erstes muss der Nutzer, welcher registriert sein muss, seinen Token generieren. Dieser Token ist 1h gültig und kann über die «Authentication API» generiert werden.

Um den Token generieren zu können, muss der Benutzer seine Login Daten als Base64 enkodierter Hash mitliefern. (username:passwort als Base64 Hash enkodiert = dXNlcm5hbWU6cGFzc3dvcnQ=)

Das Format dieser Übergabe muss folgendem Muster entsprechen:
 «Basic dXNlcm5hbWU6cGFzc3dvcnQ=»

Bei einer Token Anfrage wird der mitgesendete Hash dekodiert und in seine Einzelteile zerlegt. Die Userangaben sind in der DNASim Datenbank abgelegt und werden dort mit den mitgesendeten Angaben abgeglichen. Da das Userpasswort aus Sicherheitsgründen nicht in Klartext in der Datenbank hinterlegt ist, muss das Passwort mit der crypt() Methode und einem in der Konfigurationsdatei abgelegten Schlüssel, verschlüsselt werden.

In der Datenbank wird nach diesem verschlüsselten Passwort und dem dazugehörigen Username gesucht. Wenn diese Kombination existiert, wird ein Token generiert und zurückgesendet. Zudem wird der Token zusammen mit der errechneten Ablaufzeit in der User Tabelle eingetragen. Wenn die Kombination nicht existiert, konnte sich der User nicht authentifizieren und der Zugriff auf einen Token wird verweigert. Der Ausgang der Aktion wird geloggt.

Nach Erhalt des Tokens, kann der Benutzer eine Geräte- oder Modul Abfrage machen.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
 Wassergasse 42
 9000 St. Gallen

Mobile 079 355 16 94
 E-Mail Michael.Fehr@Swisscom.com
 Security C2

Datum: 22.04.2020
 Seiten: 63 von 129

Bei der Abfrage muss er den Token mitsenden. Dieser wird mit dem Token in der Datenbank überprüft. Wenn ein gleicher Token existiert und dieser noch nicht abgelaufen ist, ist der Token gültig und der Benutzer berechtigt eine Abfrage zu machen. Ansonsten muss zuerst ein neuer Token generiert werden.

Die Devices oder Module werden anhand der eventuell mitgegebenen Parameter ausgelesen und in das korrekte Ausgabeformat gebracht. Der Ausgang der Abfrage wird geloggt.

Damit der Aufruf der API identisch mit der des DNA Centers ist, werden auf dem virtuellen Apache Server drei Aliase eingerichtet. Auf der Abbildung 30 bis Abbildung 32 sind die originalen URL aus dem DNA Center ersichtlich.

Für die Token Generierung:

<http://michi/dna/system/api/v1/auth/token>

Method	URL
POST	https://10.44.1.241/dna/system/api/v1/auth/token

Abbildung 30: DNA Center Token URL

Für die Device Abfrage:

<http://michi/dna/intent/api/v1/network-device>

Method	URL
GET	https://10.44.1.241/dna/intent/api/v1/network-device

Abbildung 31: DNA Center Device URL

Für die Modul Abfrage:

<http://michi/dna/intent/api/v1/network-device/module>

Method	URL
GET	https://10.44.1.241/dna/intent/api/v1/network-device/module

Abbildung 32: DNA Center Module URL

Der Name meines virtuellen Servers ist «michi», dieser widerspiegelt das DNA Center.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **64** von **129**

18.1 Header

Der Header ist in jedem File enthalten. Der Autor und das Erstelldatum sind aufgeführt. Zudem fasst er die Funktionalität des Files zusammen und beinhaltet Informationen zum kommenden Code. Ausserdem wird ein Änderungsprotokoll geführt.

```
1.  /* =====
2.  *
3.  * Name:          header.php
4.  *
5.  * Autor:         Michael Fehr
6.  * Datum:         08.04.2020
7.  *
8.  * Funktion:      Der Header ist eine Einleitung in das folgende File
9.  *
10. * Änderungsprotokoll:
11. * 08.04.2020      Änderung 1
12. * 08.04.2020      Änderung 2
13. * 08.04.2020      Änderung 3
14. *
15. * =====
```

Code 1: header

18.2 config.php

In der Konfigurationsdatei werden Daten festgehalten, an welchen einfache Änderungen gemacht werden können.

Die Zugangsinformationen zur Datenbank können ändern und müssen somit einfach angepasst werden können. Zudem werden die beiden Schlüssel, welche das personalisieren der Passwortverschlüsselung und des Tokens ermöglichen hier festgehalten. Falls jemand den Code klauen würde, könnte er die Passwörter unserer Kunden dennoch nicht entschlüsseln, da der Schlüssel zur Entschlüsselung ausschliesslich in dieser Konfigurationsdatei aufgeführt ist. Auch die Ablaufzeit des Tokens muss einfach bearbeitbar sein und ist somit in der Konfigurationsdatei festgehalten. Zudem kann die URL der API, an welche die Events gesendet werden, ändern, und ist somit ebenfalls in dieser Datei festgehalten. Mit der Debugging Funktion, können bestimmte Prints gemacht werden. Diese können beim Suchen von Fehlern nützlich sein und könne ebenfalls in der Konfigurationsdatei eingeschaltet werden.

```
1.  // Folgend sind die Zugangsinformationen für die Datenbank "DNASim" festgehalten
2.  $db['User'] = 'dnasim';
3.  $db['Host'] = 'localhost';
4.  $db['Password'] = 'Labor1234';
5.  $db['Name'] = 'DNASim';
6.
7.  // Der geheime Salt, um die Passwortverschlüsselung zu garantieren
8.  $config['salt'] = 'mysalt1234';
9.
10. // Der Secret Key, welcher bei der Tokengenerierung benötigt wird.
11. $config['signature'] = 'MySecret01!';
12.
13. // Der Token ist 3600 Sekunden - 1 Stunde gültig
```

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **65** von **129**

```
14. $config['lifetimeToken'] = '3600';  
15.  
16. // Events werden an diese API gesendet  
17. $config['url'] = 'http://inventarmichi/receiveEvents';  
18.  
19. // Debug (true/false)  
20. $config['debug'] = 'true';
```

Code 2: config.php

18.3 dbFunctions.php

In dieser Datei werden alle Funktionen, welche etwas mit der Datenbank zu tun haben, abgespeichert. Dieses File benötigt Zugriff auf die erwähnte config.php-Datei, da die Datenbankverbindung nur mithilfe der Einträge aus der Konfigurationsdatei möglich ist.

18.3.1 dbConnect(array \$db) : mysqli

In der dbConnect Funktion, wird die Datenbankverbindung hergestellt und in einer Variable abgespeichert. Diese Variable ist in jeder weiteren Datenbankfunktion als Parameter mitzugeben.

```
$connect = mysqli_connect($db['Host'], $db['User'], $db['Password'], $db['Name']);
```

Code 3: \$connect

Wenn falsche Angaben verwendet werden, oder die Datenbankverbindung aus einem anderen Grund scheitert, wird dies ausgegeben.

18.3.2 dbDisconnect(mysqli \$connect) : bool

Um die Datenbankverbindung am Ende einer Sitzung wieder zu schliessen, wird die dbDisconnect Funktion aufgerufen.

Als Parameter muss die bestehende Datenbankverbindung mitgeliefert werden.

```
1. 1. function dbDisconnect($connect){  
2. 2.     mysqli_close($connect);  
3. 3. }
```

Code 4: dbDisconnect()

18.3.3 dbRead(mysql \$connect, string \$sql) : array

Mithilfe der dbRead Funktion, kann man alle Inhalte einer Tabelle auslesen. Diese Funktion wird beispielsweise beim Auslesen der Devicedetails oder auch der Userangaben verwendet.

Als Parameter muss die Zugangsberechtigung zur Datenbank, gespeichert in der \$connect-Variabel, mitgegeben werden. Zudem wird mit dem Sql Befehl die gewünschte Abfrage mitgeliefert.

```
1. 1. function dbRead($connect, $sql){
2. 2.     $result = mysqli_query($connect, $sql);
3. 3.     while($row[] = mysqli_fetch_assoc($result)) {
4. 4.     }
5. 5.     return $row;
6. 6. }
```

Code 5: dbRead()

Die Funktion liest einen Eintrag nach dem Anderen aus und schreibt diesen in ein assoziatives Array, welches auch den Rückgabewert der Funktion darstellt.

18.3.4 dbUpdate(mysql \$connect, string \$table, string \$token, string \$expireTime, int \$dnaUserId) : updatedTable

Mit der dbUpdate Funktion können bestehende Einträge aktualisiert werden. Diese Funktion wird angewendet, wenn ein neuer Token generiert wird. Der Token und dessen Ablaufzeit wird in der User Tabelle neu eingetragen.

Da im IPA Projekt nur eine Tabelle mit den immer gleichen Werten aktualisiert werden muss, können diese direkt eingegeben werden. Um eine generell gültige Funktion zu programmieren, muss man die Werte in einem Array übergeben, welche danach einzeln ausgelesen werden. Die Key-Value Paare, werden die zu aktualisierende Attribute und die einzufügenden Elemente darstellen.

Als Parameter werden die Datenbankverbindung, die Tabelle, der Token, die Ablaufzeit und die UserId verlangt.

```
1. 1. function dbUpdate($connect, $table, $token, $expireTime, $dnaUserId){
2. 2.     // Der SQL-Befehl wird zusammengebaut
3. 3.     $sql = "UPDATE $table SET token = '$token.', expireTime = $expireTime WHERE
4. 4.         id = '$dnaUserId.'";
5. 5.     // führt den SQL Befehl auf der Datenbank aus
6. 6.     return $connect->query($sql);
7. 7. }
```

Code 6: dbUpdate()

Der Sql Befehl wird direkt in der Funktion aus den Parametern zusammengebaut und ausgeführt.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **67** von **129**

Wenn die dbUpdate Funktion bei der Token Generierung nicht funktioniert, kann auch kein Token ausgegeben werden. Nachdem im Header ein «500 – Update failed» ausgegeben wurde, wird die Tokenabfrage abgebrochen.

```
1. if(dbUpdate($connect, 'users', $token, $expireTime, $dnaUserId)){  
2. } else {  
3.   header("HTTP/1.1 500 - Update failed");  
4.   break;  
5. }
```

Code 7: Update failed

18.4 getToken.php

GetToken simuliert die Tokenabfrage des DNA-Centers und liefert dem User den Token, welchen er für alle weiteren Abfragen benötigt.

Auf der Abbildung 33 ist der Ablauf der Tokenabfrage ersichtlich.

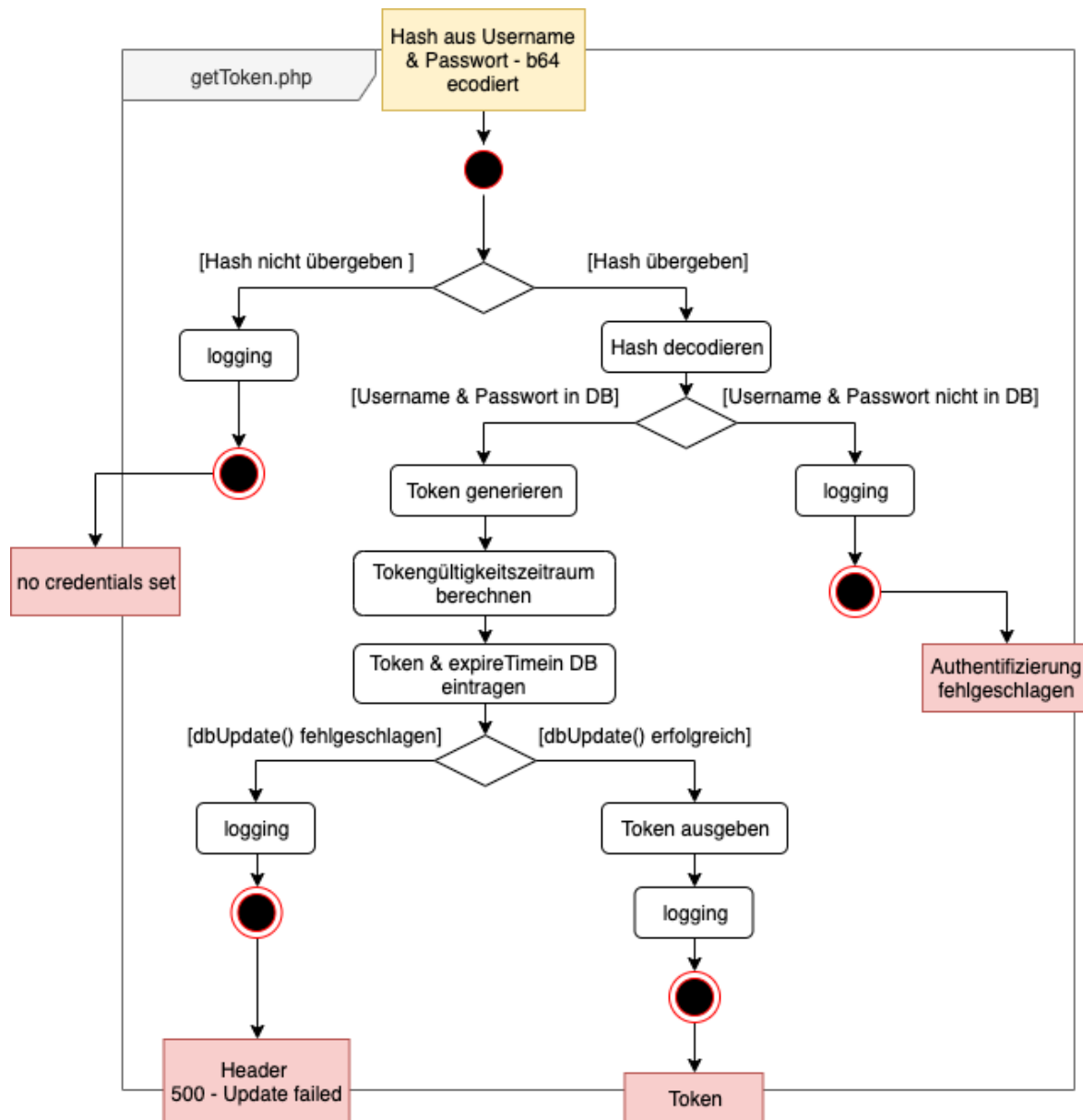


Abbildung 33: Flussdiagramm getToken.php

Um einen Token zu generieren, muss der User seine Zugangsdaten in einem festgelegten Format übergeben. Wenn der base64 kodierte Hash aus Benutzernamen und Passwort nicht übergeben wird, kann kein Token generiert werden.

Wenn der User einen Hash übergeben hat, wird dieser ausgelesen, dekodiert und in seine Einzelteile zerlegt. Im folgenden Codeausschnitt wird die bereits dekodierte Usereingabe in seine Einzelteile zerlegt.

```
$decodedHashFromUserSeparate = (explode(":", $decodedHashFromUser));
```

Code 8: \$decodedHashFromUserSeparate

Der Code 8 liefert die Output welcher bei Code 9 abgebildet ist:

```
1. 1. Array
2. (
3. 3.    [0] => dnacapi01
4. 4.    [1] => Api1234
5. 5. )
```

Code 9: Output \$decodedHashFromUserSeparate

Da in diesem Output das Passwort in Klartext festgehalten ist, muss es mit der crypt() Funktion verschlüsselt werden. Dabei kommt der eigene Sicherheitsschlüssel aus der Konfigurationsdatei, welcher den Hash nicht rückverfolgbar macht, zum Zug. (Group)

Rounds bestimmt in wie vielen Runden der Hash, nach der Feistelchiffre Verschlüsselung verschlüsselt wird. (Wikipedia, 2019)

```
$hashedPwd = crypt($userPassword, '$5$rounds=77796$'. $config['salt'].'$');
```

Code 10: crypt()

Die PHP eigene password_hash() Funktion generiert im Gegensatz zur crypt() Methode ausführlichere und sichere Hashes. Der Vorteil von password_hash() ist, dass wenn man keinen eigenen Schlüssel mitgibt, die Funktion selber einen generiert, was sehr sichere und nicht rückverfolgbare Zeichenfolgen generiert. Daher wird sie eigentlich auch empfohlen. Weil der Hash allerdings im Rahmen dieser Arbeit ein zweites Mal identisch erzeugbar sein muss, und diese trotzdem Simulation keine schützenswerten Informationen enthält, kann die crypt() Methode problemlos angewandt werden.

Crypt() generiert ein auf DES basierenden String. Der DES ist ein Verschlüsselungsstandard und wird von allen Betriebssystemen unterstützt. (Wikipedia, 2020)

Wenn die User/Passwort Kombination in der Datenbank existiert, ist der User berechtigt einen Token zu erhalten. Ist dies nicht der Fall, hat der Benutzer die falschen Zugangsdaten übermittelt oder verfügt über kein Konto. Mit der folgenden Sql Abfrage, kann die User/Passwort Kombination in der Userstabelle überprüft werden.

```
$dnaUser = dbRead($connect, "SELECT id, username, password, token, expireTime, email FROM users WHERE username='".$userName.'" and password='".$userPasswordHashed.'"");
```

Code 11: User/Passwort Kombination

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **70** von **129**

Existiert der Nutzer, wird die generateToken Funktion aufgerufen, welchen den JWT generiert.

Der generierte Token, wird in ein Array geschrieben und für die korrekte Ausgabe im JSON Format vorbereitet. Der Token ist nur 1h gültig, danach muss ein neuer generiert werden um Abfragen zu tätigen. Da die Laufzeit des Tokens im Konfigurationsfile festgehalten ist, kann diese einfach geändert werden. Die Ablaufzeit des Tokens wird vor der Aktualisierung der Datenbanktabelle ermittelt.

```
1. // Ablaufzeit des Token (jetzige Zeit + 3600s) ermitteln
2. $expireTime = time() + $config['lifetime'];
```

Code 12: Tokenlifetime

Danach wird der Token und dessen Ablaufzeit mithilfe der dbUpdate Funktion in der Users Tabelle aktualisiert. Nachdem der Token ausgegeben wurde wird ein Log Eintrag gemacht, welcher die Aktion abschliesst.

Der Codeausschnitt von Code 13 zeigt wie das korrekte Ausgabeformat erreicht wird:

```
echo json_encode($tokenArray, JSON_PRETTY_PRINT);
```

Code 13: echo Token

Die Abbildung 34 zeigt die Token Ausgabe, welche identische zu der des DNA Centers ist.

```
1
2  "Token":
3  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Ikhjc2ljIFpHNWhZMkZ3YVRBeE9rRndhVEV5STxpRPSIsImV4cGlyZVRpbWUiOiIxNTg2ODU5NjI4In0.y30GqT9DS-MAmZ_TGupbe1MdDNE58PXVBvAEVR0WIXw"
```

Abbildung 34: Token Ausgabe

Wenn alles geloggt wurde, wird zum Schluss die Datenbankverbindung mit der dbDisconnect Funktion beendet.

18.5 tokenFunctions.php

In diesem File werden alle Funktionen, welche sich auf den Token beziehen, ausprogrammiert. Die Konfigurationsdatei wie auch die Funktionen, welche mit der Datenbank agieren, müssen eingebunden werden.

18.5.1 generateToken(string \$hashFromUser, string \$dnaUserExpireTime) : string

Die Funktion benötigt zwei Parameter. Der erste Parameter stellt den Hash des Users dar, welchen er beim Aufruf der Token-API mitgeben musste. Zudem muss die Ablaufzeit des Tokens mitgesendet werden. Die Ablaufzeit wird später im Payload verwendet.

Der Token, welcher in dieser Funktion generiert wird, um künftige Abfragen zu autorisieren, hat das Format eines JSON Web Tokens. (Waller, 2018) Da auch das DNA Center diesen Token Typ verwendet, habe ich hier keinen Entscheidungsspielraum.

Der JWT besteht aus 3 Teilen, welche mit einem Punkt getrennt werden.

```
$token = $base64UrlHeader . "." . $base64UrlPayload . "." . $base64UrlSignature;
```

Code 14: JWT Zusammensetzung

Header

Im Header sind Informationen über die Art des Tokens festgehalten. Ausserdem wird hier der Hash-Algorithmus festgelegt. Im Rahmen der IPA wird der HS256 Hash-Algorithmus angewendet, welcher typisch für einen JWT ist.

```
$header = json_encode(['alg' => 'HS256', 'typ' => 'JWT']);
```

Code 15: JWT Header

Payload

Im Payload werden die Informationen zum Empfänger des Tokens, also dem User festgehalten. Das DNA Center verlangt einen Base 64 enkodierten Hash aus der Username-/Passwort-Kombination. Ausserdem wird die Ablaufzeit des Tokens mitgegeben.

```
$payload = json_encode(['username' => 'Basic ' . $hashFromUser, 'expireTime' => $dnaUserExpireTime]);
```

Code 16: JWT Payload

Signature

Die Signatur dient der Sicherheit des Tokens und besteht aus dem enkodierten Header und Payload. Zudem wird ein Sicherheitsschlüssel beigefügt, welcher die Sicherheit des JWT erhöht. Der verwendete Sicherheitsschlüssel ist in der Konfigurationsdatei abgelegt.

```
$signature = hash_hmac('sha256', $base64UrlHeader . "." . $base64UrlPayload, $config['signature'], true);
```

Code 17: JWT Signature

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **72** von **129**

18.5.2 checkToken(mysql \$connect) : bool

Da bei jedem API-Aufruf als erstes der Token geprüft werden muss, eignet sich eine Funktion hervorragend. Der Datenbankzugang und die ausgelesenen Headerinformationen müssen als Parameter mitgegeben werden.

Die Abbildung 35: Flussdiagramm checkToken() zeigt den Ablauf welcher bei der Token Überprüfung durchlaufen wird.

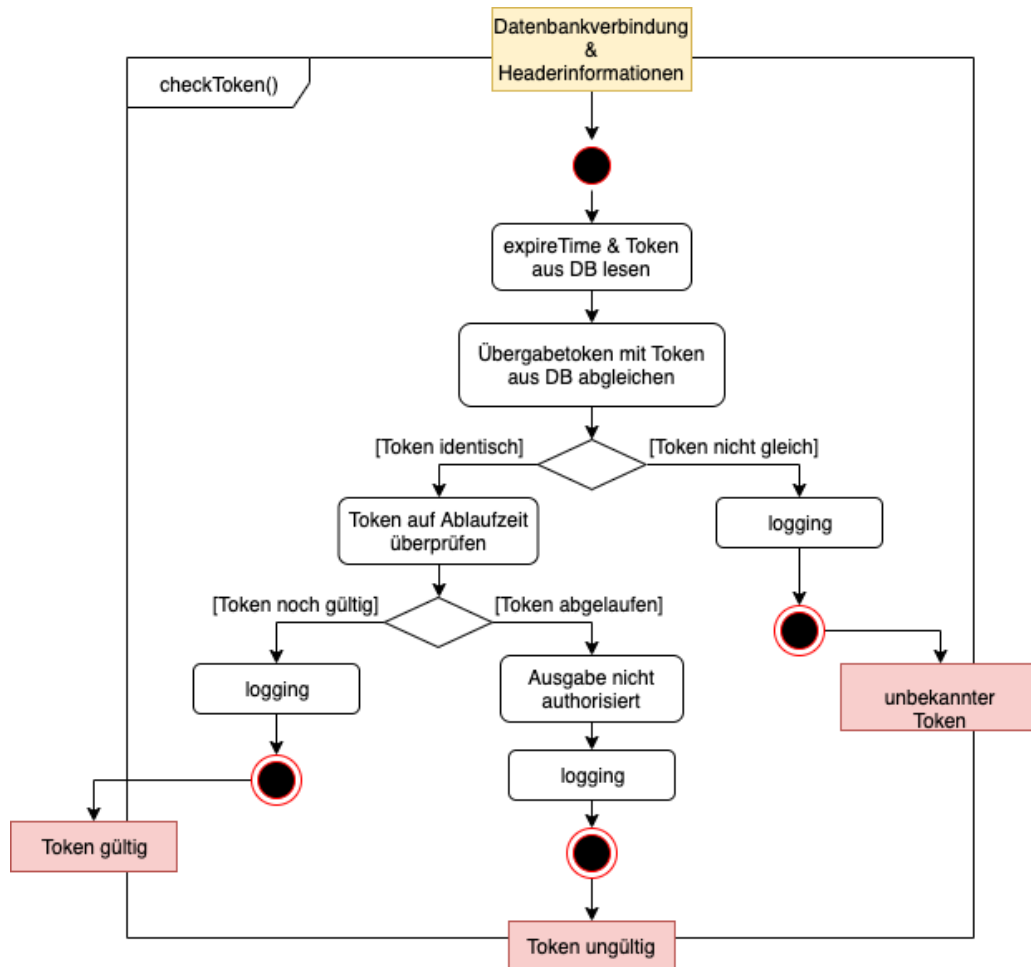


Abbildung 35: Flussdiagramm checkToken()

Mithilfe der dbRead Funktion, wird der existierende Token aus der Datenbank und dessen Ablaufzeit aus der Datenbank gelesen.

```

1. // Hier wird der Token und dessen Ablaufzeit aus der Datenbank gelesen
2. $tokenInDB = dbRead($connect, "SELECT token, expireTime FROM users WHERE id = $userId");

```

Code 18: Token in Datenbank

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **73** von **129**

Um den Token aus der User Übergabe zu überprüfen, wird dieser nun mit dem Token in der Datenbank abgeglichen.

```
$tokenRegistered = array_search($token[1],$tokenInDB[0]);
```

Code 19: Vergleichen der Token

Wenn die beiden Token nicht gleich sind, gilt die Autorisation als fehlgeschlagen. Die Aktion wird geloggt und dem User den entsprechenden Output zurückgegeben.

Bei einer Übereinstimmung der beiden Tokens, wird die Ablaufzeit des Tokens aus der Datenbank mit der aktuellen Zeit verglichen. Ist der Token aus der Datenbank noch keine Stunde alt, ist er gültig. Ansonsten ist er ungültig und der Benutzer muss zuerst einen neuen Token generieren.

```
1. if(array_search($token[1],$tokenInDB[0])){
2.     $expireTime = $tokenInDB[0]['expireTime'];
3.     // Wenn der Token noch nicht abgelaufen ist...
4.     if($expireTime > time()){
5.         // Rückgabe - Der User ist berechtigt.
6.         return true;
7.         // Wenn der Token abgelaufen ist...
8.     }else{
9.         // Output - User ist nicht berechtigt.
10.        return false;
11.    }
12. }else{
13.     // Output - User ist nicht berechtigt.
14.     return false;
15. }
```

Code 20: Token auf Ablaufzeit prüfen

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile
E-Mail
Security

079 355 16 94
Michael.Fehr@Swisscom.com
C2

Datum: 22.04.2020
Seiten: **74** von **129**

18.6 logging.php

Im Logging.php File, ist die Logging Funktion ausprogrammiert. Bei allen wichtigen Aktionen, wird ein Log Eintrag gemacht. Als wichtige Aktionen gelten Abfragen auf eine der API's. Wenn ein User beispielsweise ein Token generiert, wird ein Log Eintrag erstellt, welcher die Stufe Info hat. Wenn der User allerdings die Device Abfrage ausführt aber keinen oder einen ungültigen Access-Token mitsendet, wird dies in der Logtabelle als Warnung festgehalten.

Im Code 21 ist ersichtlich wie die Logging Funktion aufgerufen werden kann und welche Parameter benötigt werden.

```
logging($connect, date("Y-m-d H:i:s"), "", "Access attempt with invalid token", "warning", "device");
```

Code 21: Logging Aufruf

In der Funktion wird ein Sql Befehl aus den Parametern zusammengesetzt. Da ein neuer Wert in eine bestehende Tabelle eingeführt wird, wird die «Insert» Variante verwendet.

Auf den Zeilen 7 - 9 des Code 22, wird bei fehlschlagendem Ausführen des Sql Befehls, der Sql Befehl ausgegeben. So kann der Entwickler Fehler schneller und einfacher nachvollziehen.

```
1. function logging($connect, $time, $user, $logMessage, $severity, $scope){
2.     // Sql Befehl für insert in die Logtabelle wird zusammengesetzt
3.     $sql = "INSERT INTO logs (time, user, logMessage, severity, scope) VALUES ('$time'
4.         , '$user', '$logMessage', '$severity', '$scope')";
5.     // Der Sql Befehl wird ausgeführt
6.     if ($connect->query($sql)) {
7.         // Wenn der Sql-Befehl nicht ausgeführt werden konnte...
8.     } else {
9.         // Der Sql Befehl wird ausgegeben
10.        print "$sql \n";
11.    }
```

Code 22: Logging Funktion

Mit der Log Funktion hat man beinahe unbegrenzte Möglichkeiten. Wenn ich mehr Zeit zur Verfügung hätte, könnte ich weitere Zustände oder Gegebenheiten loggen.

Beispielsweise wäre es bei einer fehlgeschlagenen Autorisierung mit dem Token spannend zu sehen, was nicht stimmt: Der Token selbst oder die Ablaufzeit.

Man könnte bei einer Device oder Modul Abfrage auch die gegebenen Parameter und somit den genauen Request des Users festhalten.

18.7 getDevices.php

Beim Aufruf des getDevices.php Files, wird der Token als Parameterübergabe gefordert und sendet bei erfolgreichem Durchlaufen des Programmes die Geräte entsprechend der Abfrage als JSON. Das Flussdiagramm auf Abbildung 36 berücksichtigt die Parameterübergabe nicht, weil dies die Übersicht stark beeinträchtigen würde.

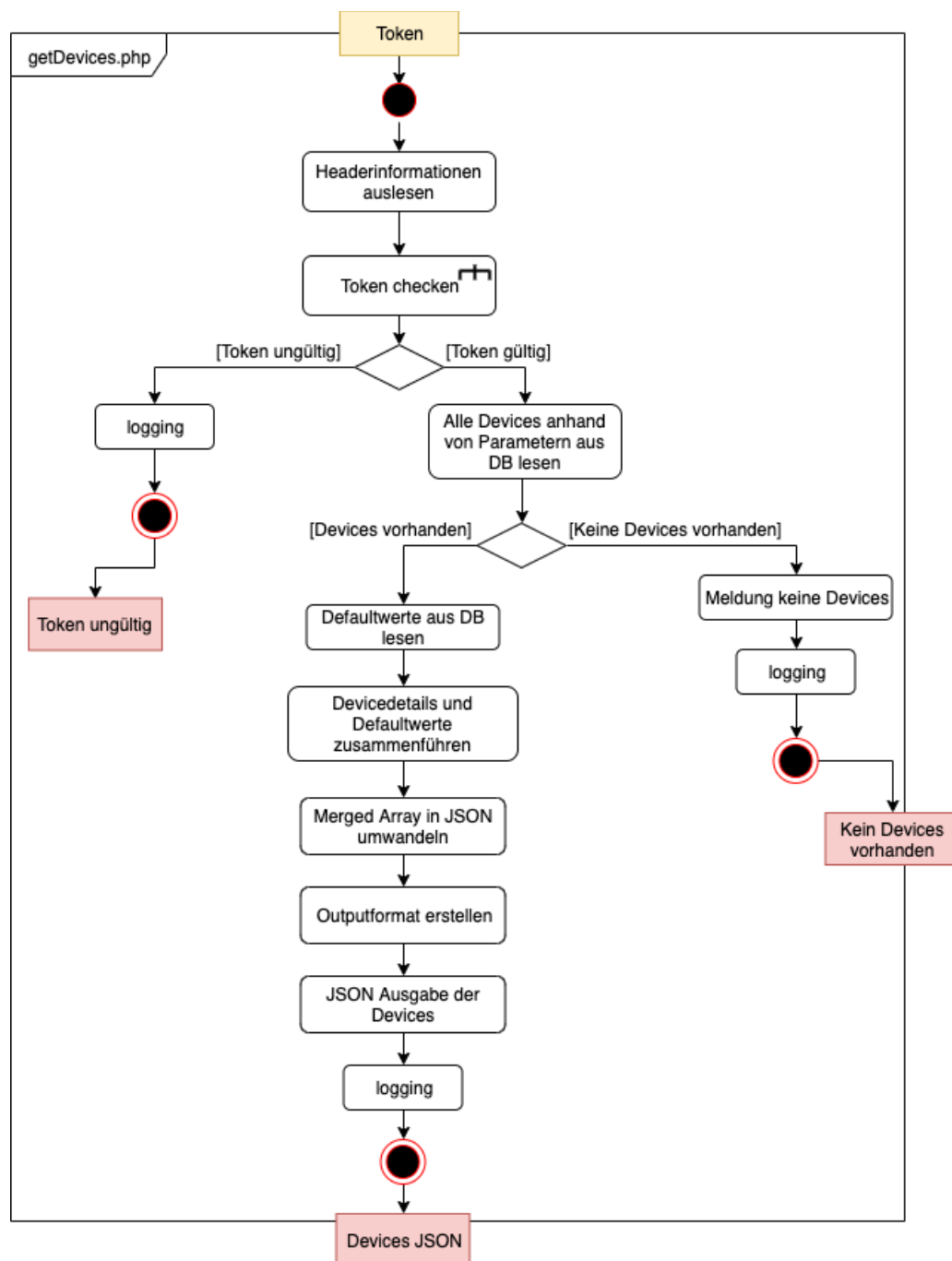


Abbildung 36: Flussdiagramm Device Abfrage

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: 76 von 129

Als erstes wird der Token, welcher der User bei der Abfrage mitgeliefert hat, aus dem Header gelesen und in eine Variable namens \$token gespeichert.

```
1. 1. $headerInformations = getAllheaders();  
2. 2. // Token aus Postman Parameter in Variable schreiben  
3. 3. $token = explode(" ", $headerInformations['Authorization']);
```

Code 23: Token aus User Übergabe

Dieser Token wird danach an die checkToken() Funktion, welche bereits erwähnt wurde, gesendet. Wenn der Token gültig ist, wird der Wert «true» wiedergegeben.

Um alle möglichen Parameter behandeln zu können, wird ein Array mit allen Parametern erstellt, welche allerdings noch keine Werte enthalten und auf Code 24 ersichtlich ist.

```
1. $myParameters = array(  
2.     "hostname" => "",  
3.     "managementIpAddress" => "",  
4.     "macAddress" => "",  
5.     "locationName" => "",  
6.     "serialNumber" => "",  
7.     "location" => "",  
8.     "family" => "",  
9.     "type" => "",  
10.    "series" => "",  
11.    "collectionStatus" => "",  
12.    "collectionInterval" => "",  
13.    "notSyncedForMinutes" => "",  
14.    "errorCode" => "",  
15.    "errorDescription" => "",  
16.    "softwareVersion" => "",  
17.    "softwareType" => "",  
18.    "platformId" => "",  
19.    "role" => "",  
20.    "reachabilityStatus" => "",  
21.    "upTime" => "",  
22.    "associatedWlcIp" => "",  
23.    "id" => ""  
24.);
```

Code 24: \$myParameters Device

Zudem wird ein leeres Array erstellt, welche die gesetzten Parameter empfangen wird.

Als nächstes werden alle Parameter anhand des Arrays von Code 24 einzeln überprüft. Wenn die DeviceId als Parameter gesetzt ist, werden keine weiteren Parameter berücksichtigt, auch wenn diese falsch sind. Die DeviceId ist entscheidend.

```
1. if(isset($_REQUEST['id'])){
2.     $sqlTagId = " where id='".$_REQUEST['id']."'";
```

Code 25: SqlTagId

```
$sqlBasic = "SELECT * from device_detail";
```

Code 26: Sql Basic Device

Wenn die Id nicht in den gesetzten Parametern ist, werden die Parameter wie folgt in das Array aller Parameter gespeichert.

```
$myParameters[$parameter] = $parameter."='".$_REQUEST[$parameter]."'";
```

Code 27: \$myParameters füllen

Wenn die Liste der Parameter am Schluss nicht leer ist, werden die gesetzten Parameter in das noch leere Array der gesetzten Parameter geschrieben.

```
1. if (!empty($myParameters[$parameter])){
2.     array_push($setParameters, $myParameters[$parameter]);
3. }
```

Code 28: \$setParameters füllen

Nun sind alle gesetzten Parameter in einem Array festgehalten. Jetzt kann der SQL String unter Berücksichtigung aller Parameter zusammengesetzt werden, um die korrekte Abfrage zu ermöglichen.

Da die Parameter bei Code 27 Bereits im richtigen Format eingetragen wurden, können sie nun nur mit der korrekten Trennung hintereinander gesetzt werden. Da die Einträge mit «AND» getrennt werden, wird der Sql Befehl später nur Ergebnisse liefern, wenn alle Parameter kombiniert werden können.

```
$sqlTagAdditional = implode(" AND ", $setParameters);
```

Code 29: sqlTagAdditional Device

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **78** von **129**

Je nach gesetzten Informationen wird der zu verwendende Sql Befehl zusammengesetzt und an die dbRead() Funktion übergeben. Diese wird allerdings nur ausgeführt, wenn die Token Überprüfung den Wert «true» zurückgegeben hat. Ansonsten wird der Prozess hier beendet, da sich der Benutzer mit dem Token nicht autorisieren konnte.

Nachdem die gewünschten Devices ausgelesen wurden und überprüft wurde, ob diese Ausgabe auch ein Device enthält, werden die Default Werte aus der Datenbank gelesen. Wenn kein Device anhand der gewünschten Abfragekriterien gefunden wurde, wird dies ausgegeben und in die Logtabelle eingetragen.

Im folgenden Codeabschnitt werden die Werte, welche nicht aus dem DNA Center geliefert werden, herausgefiltert. Zudem werden einzeln zu jedem Device die Default Werte hinzugefügt.

```
1. foreach($devices as $device){
2.     // Elemente, welche nicht aus dem DNA Center kommen entfernen
3.     unset($device['fk_device_type_id']);
4.     unset($device['typeId']);
5.     // Für jeden Default Wert...
6.     foreach($defaults[0] as $key => $value){
7.         // Device wird mit den Default Werten ergänzt
8.         $device[$key] = $value;
9.     }
10.    // Jedes einzelne Device mit den ergänzten Werten wird in das $devicesComplete
    Array gelegt
11.    array_push($devicesComplete, $device);
12. }
```

Code 30: unset values Device & push defaults

Das zusammengeführte Array wird nun noch in das korrekte Format gebracht und ausgegeben.

```
1. // Das Array wird hier zur Ausgabe vorbereitet, sprich JSON encoded
2. $intent_json_encoded = json_encode($devicesComplete);
3. // Ausgabe der devices im korrekten Format
4. $networkDevicesOutput = '{"response":'.$intent_json_encoded.', "version": "1.0"}';
5. // Devices Output
6. echo $networkDevicesOutput;
```

Code 31: Ausgabe Devices

Nachdem die Ausgabe erfolgte wird dies geloggt und zum Schluss die Datenbankverbindung beendet.

18.8 getModules.php

Das getModules.php File simuliert die Handhabung einer Module Abfrage. Der Token muss zur Autorisierung mitgesendet werden. Um den Token zu überprüfen, wird dieser aus dem Header ausgelesen und mit der checkToken() Funktion auf die Gültigkeit geprüft. Das Flussdiagramm von Abbildung 37 zeigt den Ablauf einer Modul Abfrage, wobei die genaue Abfolge der Parameterbehandlung ausgelassen wird, da sie unten genauer aufgefasst wird.

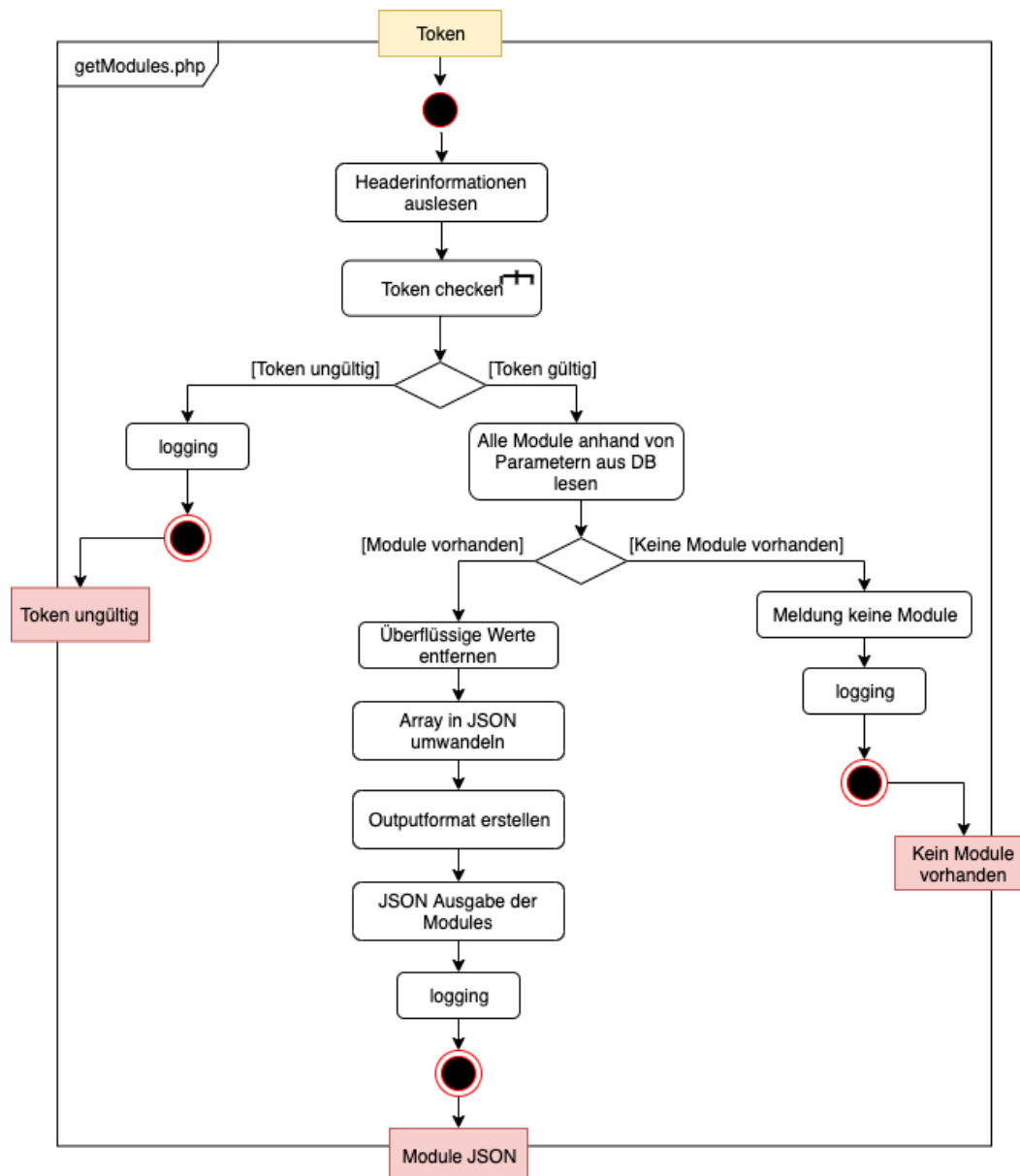


Abbildung 37: Flussdiagramm Modul Abfrage

Der Ablauf der Token Überprüfung läuft identisch wie bei der Device Abfrage ab. Bei Code 23 kann dieser erneut eingesehen werden.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **80** von **129**

Wichtig zu wissen ist, dass ein Modul niemals alleine existieren kann. Ein Modul ist immer einem Device untergeordnet und kann somit nur über die DeviceId aufgerufen werden, welche zwingend als Parameter übergeben werden muss. Wird diese nicht mitgeliefert, wird die Abfrage keinen Output der Module liefern können. Der Vorgang wird geloggt.

```
1. / Auslesen der deviceId aus User Parameter
2. $deviceId = $_REQUEST['deviceId'];
3. // Wenn die deviceId nicht gesetzt wurde...
4. if(empty($deviceId)){
5.     echo "400 - deviceId is missing or invalid";
```

Code 32: check deviceId

Wie bei den Parametern der Device Abfrage, wird ein Array erstellt. Allerdings sind in diesem Array nicht alle Parameter, welche zur Verfügung stehen enthalten. Dies wird durch die logische Zusammensetzung des Sql Befehles begründet.

```
1. // Alle Parameter ausser Limit und Offset in einem Array
2. $myParameters = array(
3.     "name" => "",
4.     "vendorEquipmentType" => "",
5.     "partNumber" => "",
6.     "operationalStateCode" => ""
7. );
```

Code 33: \$myParameters Module

Alle Parameter in \$myParameters, werden einzeln ausgelesen und mit der gleichen Logik wie bei der Device Abfrage in ein Array mit den gesetzten Parametern geschrieben.

Da bei einer Abfrage mit Parametern alle Parameter stimmen müssen, werden die Einträge in dem Array mit Hilfe des Sql Attributes «AND» zusammengeführt.

```
$sqlTagAdditional = implode(" AND ", $setParameters);
```

Code 34: sqlTagAdditional Module

Die beiden weiteren Parameter (limit & offset) wurden absichtlich nicht mit den anderen Parametern zusammen ausgelesen. Der Grund ist die Erstellung des Sql Befehls. Zudem kann der Offset Parameter nur gesetzt werden, wenn der Limit Parameter gesetzt wurde. Falls die Parameter gesetzt wurden, werden sie in ein eigenes Array geschrieben. Die Einträge können aus dem Array gelesen werden und mit einem anderen Trennzeichen zusammengesetzt werden. Somit entsteht der Sql Befehl, wenn das Limit und/oder der Offset mitgegeben wurde.

```
$sqlTagAdditionalOther = implode(" ", $setOtherParameters);
```

Code 35: \$sqlTagAdditionalOther

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile
E-Mail
Security

079 355 16 94
Michael.Fehr@Swisscom.com
C2

Datum: 22.04.2020
Seiten: **81** von **129**

Das Herzstück der Modulabfrage mit den angegebenen Parametern, stellt die Zusammensetzung des korrekten Sql Befehles dar. Als erstes wird der Basic Befehl erstellt. Dieser ist bei jeder Abfrage gleich.

```
$sqlBasic = "SELECT * from module_detail where fk_device_id='$deviceId';"
```

Code 36: Sql Basic Modul

Danach wird mit dem Code 37 geprüft, ob die einzelnen Parameter gesetzt wurden. Die verschiedenen Parameterzusammensetzungen geben verschiedene Sql Befehle aus, wobei der zu verwendende Befehl in der \$sqlTag Variable hinterlegt wird.

```
1. // Wenn ein Parameter gesetzt wurde (ausser Limit und Offset)...
2. if(!empty($sqlTagAdditional)){
3.     // Der SqlTag wird zusammengesetzt
4.     $sqlTag = $sqlBasic." AND ".$sqlTagAdditional;
5.     // Wenn zusätzlich Limit und/oder Offset gesetzt wurden...
6.     if(!empty($sqlTagAdditionalOther)){
7.         // Der Sql-Tag wird erweitert
8.         $sqlTag = $sqlBasic." AND ".$sqlTagAdditional.$sqlTagAdditionalOther;
9.     }
10. // Wenn kein Parameter gesetzt wurde (ausser Limit und Offset)
11. } else {
12.     // Wenn Limit und/oder Offset gesetzt wurde...
13.     if(!empty($sqlTagAdditionalOther)){
14.         // Der Sql-Tag wird zusammengesetzt
15.         $sqlTag = $sqlBasic.$sqlTagAdditionalOther;
16.         //Wenn gar keine Parameter gesetzt wurden
17.     } else {
18.         // Der Sql-Tag ist = der Sal-Basic String
19.         $sqlTag = $sqlBasic;
20.     }
21. }
```

Code 37: Sql Befehl Zusammensetzung

Mit dem generierten Sql Befehl wird die dbRead Funktion aufgerufen und die passenden Module ausgelesen. Es werden wiederum die Daten, welche nicht vom DNA Center geliefert werden, entfernt.

```
1. for($i = 0; $i<=sizeof($modules); $i++){
2.     unset($modules[$i]['fk_device_id']);
3.     unset($modules[$i]['fk_module_type_id']);
4. }
```

Code 38: unset values Module

Wie bei jeder Abfrage, werden die Daten in das korrekte Format gebracht, ausgegeben und zum Schluss geloggt.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **82** von **129**

18.9 events.php

Das events.php File gibt simulierte Alarmmeldungen an eine bestehende API aus. Die URL der API, ist im Konfigurationsfile hinterlegt. Events alarmieren den Kunden beispielsweise über ausgefallene Router oder aussergewöhnlich hohe Auslastung von Geräten. Die vordefinierten Eventtypen sind in der DNASim Datenbank hinterlegt und auf der Abbildung 28 einsehbar. Um einen Event zu senden, ruft man das File mithilfe des «-c» Parameter auf. Wenn man den «-c» Parameter nicht mitgibt, wird eine Option ausgeführt, welche als Zusatzarbeit erledigt wurde und bei Punkt 18.9.1 auf Seite 84 einsehbar ist. Man wird auf der Kommandozeile aufgefordert einige Informationen mitzugeben, sofern man diese noch nicht als Parameter mitgegeben hat. Neben dem Event Typ muss, wie in den Kriterien gefordert, auch der Alarmtext und der Schweregrad mitgegeben werden können. Mit «-x» kann der Debugging Modus eingeschaltet werden, welcher wichtige Informationen ausgibt. Wenn Hilfe benötigt wird, kann diese mit «-h» aufgerufen werden.

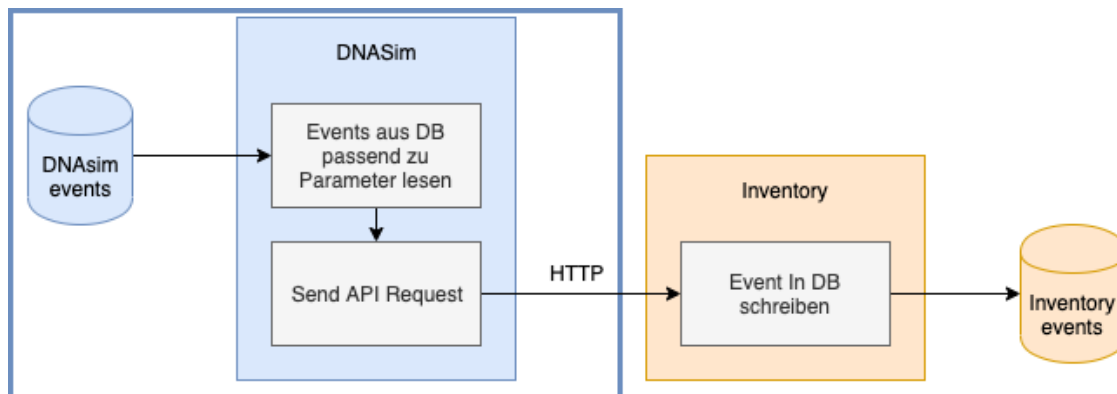


Abbildung 38: Gesamtübersicht Events

Bei erfolgreichem senden eines Events, wird dieser in der virtuellen Maschine, welche die Seite des Users darstellt, in die Datenbank geschrieben. (Auf der Abbildung 38 in orange)

Der Code 39 erzeugt eine Tabelle, auf Abbildung 39 ersichtlich, welche dem Benutzer die Auswahl aller Event Typen auflistet. Da dem Nutzer nicht zugemutet werden kann, dass er einen Event Typ ohne Fehler eintippt, kann er ihn mit der dazugehörenden Nummer anwählen.

```

1. $table = "|%3.5s |%-30.30s |%-50.50s |\n";
2. foreach($allEventIds as $id => $eventId) {
3.     printf($table, $id, $allEventIds[$id], $allNames[$id]);
4. }

```

Code 39: Table Output Terminal

```

Dear user,
You have to choose which Eventtype you want to send.
Please choose one of the following Eventtypes by number:

| 0 | NETWORK-DEVICES-2-152 | WLC Reboot Crash
| 1 | NETWORK-DEVICES-2-201 | Switch Power Failure
| 2 | NETWORK-DEVICES-2-202 | Device Reboot Crash
| 3 | NETWORK-DEVICES-2-204 | Stack Member Running Incompatible Image
| 4 | NETWORK-DEVICES-2-205 | Switch Fan Failure
| 5 | NETWORK-DEVICES-3-104 | AP Memory High Utilization
| 6 | NETWORK-FABRIC_WIRED-1-310 | Fabric Devices Connectivity Control Underlay
| 7 | NETWORK-NON-FABRIC_WIRED-1-250 | Router unreachable

```

Abbildung 39: Table in Terminal

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **83** von **129**

Aus allen Parametern und/oder nachträglichen Eingaben des Benutzers, wird ein Sql Befehl generiert. Durch das Ausführen dieses Befehles, bekommt man den korrekten Event. Dieser wird JSON enkodiert und über die URL an die API gesendet. (Code 40)

```
1. // Der Übergabewert wird JSON enkodiert.
2. $payload = json_encode($eventOutput);
3. // Der Übergabewert der POST Methode anfügen
4. curl_setopt($ch, CURLOPT_POSTFIELDS, $payload);
5. // Der content type, wird im Header auf application/JSON gesetzt
6. curl_setopt($ch, CURLOPT_HTTPHEADER, array('Content-Type:application/json'));
7. // EFührt den POST aus
8. $result = curl_exec($ch);
```

Code 40: send to Event API

18.9.1 Zusatzfunktion

Zusätzlich, wird alle 15 Minuten mit Hilfe eines Cronjobs ein Event gesendet. Der Event wird basierend auf dem Zufallsprinzip zusammengesetzt. Dabei ist es wichtig zu wissen, dass es keine Rolle spielt, ob die Eventzusammensetzung einen Sinn ergibt. (Ein kritischer Event Typ, kann auch mit dem Schweregrad «Info» ausgegeben werden und eine komplett andere Detailbeschreibung enthalten.)

Im Code 41 ist aufgezeigt, wie eine zufällige Detailbeschreibung für den Event ausgelesen wird.

```
1. $keyDetail = array_rand($allDetails);
2. $details = $allDetails[$keyDetail];
```

Code 41: random detail for event

Es wird ein Logfile des Cronjobs geführt. Bei jeder Ausführung wird in das File cronjobEvents.log ein Eintrag gemacht. Der Eintrag enthält die Ausführungszeit des Cronjobs.

```
1. // Hier wird jede Ausführung des Cronjobs festgehalten
2. $fp = fopen("cronjobEvents.log", "a");
3. fwrite($fp, "cronjob running time: ".time()."\n");
4. fclose($fp);
```

Code 42: Erstellung Logfile Cronjob

Zusätzlich wird ein Shell Skript namens sendEvents.sh verwendet. Dieses Skript sorgt für die Vergabe der korrekten Pfade.

```
1. #!/bin/bash
2. cd /var/www/ipa_dna_simulator/events
3. /usr/bin/php sendEvents.php
```

Code 43: sendEvents.sh

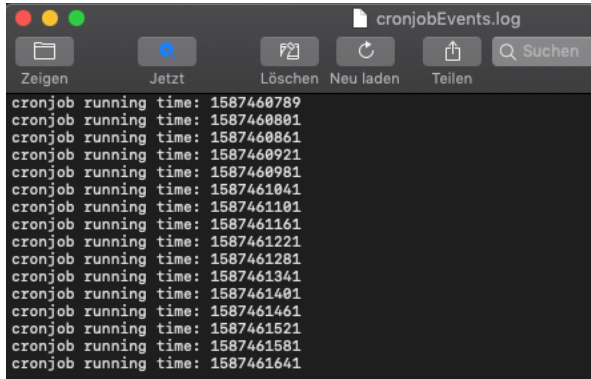
Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **84** von **129**

Für die Veranschaulichung der Logeinträge vom Cronjob, wurde der Cronjob kurzzeitig jede Minute ausgeführt. Dies generierte den Output welcher auf Abbildung 40 ersichtlich ist.



```
cronjobEvents.log
Zeigen Jetzt Löschen Neu laden Teilen Suchen
cronjob running time: 1587460789
cronjob running time: 1587460801
cronjob running time: 1587460861
cronjob running time: 1587460921
cronjob running time: 1587460981
cronjob running time: 1587461041
cronjob running time: 1587461101
cronjob running time: 1587461161
cronjob running time: 1587461221
cronjob running time: 1587461281
cronjob running time: 1587461341
cronjob running time: 1587461401
cronjob running time: 1587461461
cronjob running time: 1587461521
cronjob running time: 1587461581
cronjob running time: 1587461641
```

Abbildung 40: Logfile Cronjob

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **85** von **129**